Two-Tier E-Commerce Application on Linux

DevOps Homelab | Deployment, Security, Troubleshooting and Validation

Project status: Manual deployment complete; application and product data verified after reboot.

Platform: Three CentOS Stream 10 VirtualBox VMs; Apache HTTP Server, PHP-FPM, MariaDB, firewalld, SELinux, systemd.

Application source: https://github.com/kodekloudhub/learning-app-ecommerce

Scope: Lab-only HTTP deployment. Not a production security blueprint.

1. Project overview

Deploy a PHP e-commerce application on a dedicated web VM, with MariaDB on a separate database VM and traffic routed through a third VM. Confirm access from a Windows host, database-backed rendering, least-privilege access, firewall controls, SELinux enforcement and restart persistence.

Observed outcome: Windows successfully opened `http://192.168.56.10`; the application displayed product listings. After reboot, Apache and PHP-FPM were active, localhost returned HTTP 200, product names were present in the response, and the Windows browser continued to work.

2. Architecture and addressing

```
Windows host (browser)
       |  VirtualBox host-only network 192.168.56.0/24
       v
lab-server-1 [WEB]
  enp0s9 192.168.56.10/24 (host-only)
  enp0s3 192.168.10.10/24 (lab-net-a)
  Apache :80 -> PHP-FPM -> PHP mysqli
       |  TCP/3306; routed between internal subnets
       v
lab-router [ROUTER]
  192.168.10.1 (lab-net-a)
  192.168.20.1 (lab-net-b)
  192.168.56.30 (host-only)
  NAT interface 10.0.5.15
       |
       v
lab-server-2 [DATABASE]
  enp0s3 192.168.20.10/24 (lab-net-b)
  enp0s9 192.168.56.20/24 (host-only)
  MariaDB 10.11 on 192.168.20.10:3306
  ecomdb.products (8 rows)
```

Web tier default gateway: `192.168.10.1`; DB tier default gateway: `192.168.20.1`. IPv4 forwarding and firewalld interzone routing were configured on the router. Inter-subnet connectivity, SSH/SCP, and remote MariaDB access were tested during setup.

3. Database tier: lab-server-2

MariaDB 10.11.18 was installed and started. The `ecomdb` database uses `utf8mb4_unicode_ci`. An application account `ecom_app` was restricted to source host `192.168.10.10` and granted only `SELECT`, `INSERT`, `UPDATE`, and `DELETE` on `ecomdb.*`; DDL privileges were not granted.

The SQL seed script `assets/db-load-script.sql` was copied from the cloned repository to the DB server using SCP and imported by an administrator. The `products` table was confirmed to contain eight sample products.

MariaDB was bound specifically to `192.168.20.10:3306` using `/etc/my.cnf.d/99-ecommerce-network.cnf`. A firewalld rich rule allowed TCP 3306 only from `192.168.10.10/32` to `192.168.20.10/32`. The web VM's MariaDB CLI successfully connected to and queried the database across the router.

Representative verification commands (do not paste credentials into a shell history or documentation):

```
sudo systemctl status mariadb --no-pager
sudo ss -lntp | grep 3306
sudo mariadb -e 'SELECT COUNT(*) FROM ecomdb.products;'
sudo firewall-cmd --permanent --zone=public --list-rich-rules
```

Secret handling: The database password is intentionally omitted. A stronger password rotation was discussed, but its completion was not independently established by the recorded evidence; verify before treating the environment as secure.

4. Web tier: lab-server-1

Installed Apache HTTP Server 2.4.63, PHP 8.3.33, `php-mysqlnd`, and Git. Application source was cloned to `/home/osboxes/learning-app-ecommerce` and deployed to Apache's document root `/var/www/html/`.

The application uses `getenv('DB_HOST')`, `getenv('DB_USER')`, `getenv('DB_PASSWORD')`, and `getenv('DB_NAME')` followed by `mysqli_connect(...)` and a query against `products`. The README's suggested `.env` file is not automatically loaded by this application's PHP code.

PHP-FPM pool environment variables were configured in `/etc/php-fpm.d/99-ecommerce.conf`, with the password kept outside the web document root. The file was configured root-owned and mode 0600. The actual password is **not** reproduced here.

```
[www]
env[DB_HOST] = 192.168.20.10
env[DB_USER] = ecom_app
env[DB_NAME] = ecomdb
env[DB_PASSWORD] = <SET_PRIVATELY_ON_SERVER>
```

The PHP-FPM configuration passed `sudo php-fpm -t`, PHP-FPM restarted successfully, and the deployed application subsequently retrieved products from the remote database.

```
sudo php-fpm -t
sudo apachectl configtest
sudo systemctl enable httpd
sudo systemctl enable php-fpm
systemctl is-enabled httpd php-fpm
systemctl is-active httpd php-fpm
```

Important: Avoid copying `.git` or credential-bearing files into the publicly served document root. Review file ownership, PHP source exposure risks, directory listing, and Apache configuration before any internet-facing deployment.

5. SELinux and firewall security

SELinux remained in `Enforcing` mode. The application files initially inherited an incorrect `user_home_t` SELinux label after deployment with `cp -a`. Labels were restored to the expected `httpd_sys_content_t` using:

```
sudo restorecon -RFv /var/www/html
ls -ldZ /var/www/html
ls -lZ /var/www/html/index.php
```

Apache/PHP needed permission to connect to the remote database under SELinux. This was configured persistently:

```
sudo setsebool -P httpd_can_network_connect_db on
getsebool httpd_can_network_connect_db
# httpd_can_network_connect_db --> on
```

Both `enp0s3` and `enp0s9` on the web VM were in firewalld's `public` zone, which initially allowed SSH but not HTTP. Rather than enabling HTTP broadly, a rich rule restricted inbound TCP 80 to traffic from the host-only subnet directed at the web host-only IP:

```
sudo firewall-cmd --zone=public --add-rich-rule='rule family="ipv4" source
address="192.168.56.0/24" destination address="192.168.56.10/32" port port="80" protocol="tcp"
accept'
sudo firewall-cmd --runtime-to-permanent
sudo firewall-cmd --permanent --zone=public --list-rich-rules
```

The saved rule was verified after configuration. It permits the specified host-only subnet, not just a single Windows host.

6. Actual troubleshooting log

Issue A — Apache returned HTTP 403

Symptom: `curl -I` against `127.0.0.1` and `192.168.10.10` returned HTTP 403.

Evidence: Apache error log included `AH01276`, reporting an empty `/var/www/html` and no matching directory index; directory listing was forbidden.

Resolution: A temporary `index.html` was created to prove Apache could serve content. HTTP 200 confirmed the diagnosis. The temporary file was removed once the PHP application was deployed.

Lesson: Inspect error logs and the configured document root before changing firewall rules or permissions.

Issue B — Wrong SELinux context after copying application files

Symptom: `cp -a` preserved a `user_home_t` context on files copied from a home directory into `/var/www/html`.

Evidence: `ls -ldZ` showed the unexpected context.

Resolution: `sudo restorecon -RFv /var/www/html` restored the appropriate web-content labels; verified with `ls -lZ`.

Lesson: Linux DAC permissions and SELinux MAC labels are separate controls. Do not disable SELinux or apply `chmod 777` as a shortcut.

Issue C — MariaDB network configuration syntax

Symptom: An initial MariaDB configuration file was malformed.

Evidence: `mariadbd --print-defaults` exposed the problem before a service restart.

Resolution: Corrected the configuration and then restricted the server to `192.168.20.10:3306`.

Lesson: Validate configuration before restarting a working service.

Issue D — Application database environment

Finding: PHP used `getenv(...)`, while the repository did not load the suggested `.env` automatically.

Resolution: Supplied environment variables through a protected PHP-FPM pool configuration, tested syntax, restarted PHP-FPM and confirmed rendered product names.

Lesson: Trace how the application actually reads configuration rather than assuming README examples are implemented.

Issue E — HTTP blocked by firewalld for Windows browser

Symptom: The web server worked locally, but the public zone initially listed only `cockpit`, `dhcpv6-client`, and `ssh` services; no HTTP port or rich rule.

Resolution: Added a destination- and source-scoped rich rule for TCP 80, persisted it, and tested from Windows.

Lesson: Local HTTP 200 does not prove client reachability; inspect interfaces, listening sockets, routing and firewall zones.

Non-fatal messages

Apache emitted `AH00558` (no global ServerName) but `apachectl configtest` returned `Syntax OK`. Systemd also logged an unset `OPTIONS` variable. Neither prevented successful deployment. A `WARNING` found by grep in the HTML was an ordinary Respond.js comment, not a database error.

7. Validation evidence and commands

Before reboot: `curl` returned HTTP 200; saved HTML contained product names including Laptop, Drone, Tablet, Watch, Phone Covers and VR; no database connection, access-denied, connection-refused or fatal error text was found. Windows browser displayed the application.

After reboot: `systemctl is-active httpd php-fpm` returned `active` twice. Local `curl` returned HTTP 200. The captured HTML again contained the product names, the database error grep produced no output, and the Windows browser still displayed the application and products.

```
systemctl is-active httpd php-fpm
curl -sS -o /tmp/ecommerce-after-reboot.html -w 'HTTP Status: %{http_code}\n' http://127.0.0.1/
grep -niE 'database connection error|access denied|connection refused|fatal error' /tmp/
ecommerce-after-reboot.html
grep -oE 'Laptop|Drone|Tablet|Watch|Phone Covers|VR' /tmp/ecommerce-after-reboot.html | sort -u
```

Temporary files: `/tmp/ecommerce-after-reboot.html` is a saved HTTP response created by `curl -o`; it is **not** the deployed source. Source files reside in `/var/www/html/`. `/tmp` may be cleaned automatically.

8. Operations and security checklist

- [x] Confirm cross-subnet routing and remote database query
- [x] Use restricted MariaDB application privileges and source host
- [x] Restrict database listening address and TCP 3306 source
- [x] Keep SELinux enforcing and correct application labels
- [x] Permit Apache/PHP database access using a persistent SELinux boolean
- [x] Limit HTTP ingress to host-only network
- [x] Enable Apache and PHP-FPM at boot
- [x] Verify rendered database-backed content after web VM reboot
- [x] Confirm Windows browser access after reboot
- [ ] Verify strong credential rotation and establish secret rotation process
- [ ] Resolve duplicated `/etc/machine-id` inherited from cloned VMs
- [ ] Add HTTPS/TLS for any broader exposure
- [ ] Add monitoring, health checks, backups and recovery testing
- [ ] Automate provisioning and deployments (Ansible/Terraform/CI)
- [ ] Add production-grade least-privilege and deployment hardening review

9. Interview questions and concise answers

Q1. Why separate web and database tiers?

To isolate responsibilities and security boundaries, restrict database exposure and allow independent scaling or maintenance.

Q2. Does HTTP 200 prove database health?

No. Check rendered content, application errors and database queries. Here we verified product names, not just the status code.

Q3. Why was Apache returning 403?

The document root was empty and had no permitted index file; Apache's `AH01276` log identified the cause.

Q4. What is the difference between `systemctl start` and `enable`?

`start` launches a service now; `enable` configures boot-time startup. We verified both services after reboot.

Q5. Why use `restorecon`?

It reapplies SELinux file labels from policy. This corrected home-directory labels accidentally preserved during file copying.

Q6. Why set `httpd_can_network_connect_db`?

SELinux can restrict web processes from initiating DB connections even when IP routing and firewalls allow them.

Q7. Why use a firewalld rich rule rather than `--add-service=http`?

A rich rule allowed a source/destination-scoped exception instead of permitting HTTP from all networks in the shared public zone.

Q8. What did PHP-FPM do?

It executed PHP requests passed from Apache and supplied the configured environment variables used by the application.

Q9. Why are HTML files in `/tmp`?

They are diagnostic snapshots saved by `curl -o`, not deployed application files.

Q10. What would you improve next?

Automate repeatable provisioning, add TLS and health checks, verify secret rotation, address clone identity duplication, and implement backups and monitoring.

10. Suggested GitHub structure

```
my-devops-journey/
  linux-fundamentals/
    projects/
      two-tier-ecommerce-lab/
        README.md
        docs/
          Two_Tier_Ecommerce_DevOps_Lab_Guide.pdf
```

Do not commit database credentials, private SSH keys, or unredacted config exports. This README describes a verified homelab exercise, not a claim of production readiness.